DEEP LEARNING — 046211
FINAL EXAM B
19/3/2025

EXAM DURATION: 3 HOURS

INSTRUCTIONS.
1. You can use a calculator and the 2 formula sheets (4 pages).
2. You need to answer all question parts, unless you received a special permit.
3. The weight of each question is written in the form. The total number of points is 100 points.
4. If you received a special permit (due to reserve duty) you can choose to answer only two questions out of three. In that case, the questions points will be normalized so the total number of points will remain 100.
5. Please write clearly and orderly. You can write either in English or in Hebrew.
6. You must give the full details of the solution derivation. Solutions without explanations will not be given a score, unless written otherwise.
7. Once you finish the exam, you need to use gradescope to upload your exam. In addition, you need to submit the notebook.

1. "Typical" Generalization in Multilayer Neural Networks (30 pts)

We examine a "student" neural network $f_{\mathbf{w}}(\mathbf{x})$ with parameter vector $\mathbf{w} \in \mathbb{R}^k$ and input $\mathbf{x} \in \mathbb{R}^{d_0}$ used in a binary classification problem where the training set is $\mathcal{S} = \left\{\mathbf{x}^{(n)}\right\}_{n=1}^{N}$ sampled i.i.d. from P_X, where the binary (± 1) labels are generated by a "teacher" neural network $f_{\mathbf{w}_\star}(\mathbf{x})$ with the same architecture. To understand the "typical" generalization of the student, we examine the following "Guess and Check" algorithm to learn its weights: we randomly sample parameters vectors $\mathbf{w}_1, \mathbf{w}_2, ...$ i.i.d. from P_W, in which each parameter is sampled independently from a uniform distribution over $Q = \{-(q-1)/2, \ldots, -1, 0, 1, \ldots, (q-1)/2\}$ quantization levels, where $q = |Q|$ is an odd positive number. We do this until a stopping time t in which we perfectly fit the dataset: $\forall n : f_{\mathbf{w}_t}\left(\mathbf{x}^{(n)}\right) = f_{\mathbf{w}_\star}\left(\mathbf{x}^{(n)}\right)$. We examine a two-layer neural network with d_1 hidden neurons

$$f_{\mathbf{w}}(\mathbf{x}) = \operatorname{sign}\left(\mathbf{w}_2^\top \left[\mathbf{W}_1 \mathbf{x}\right]_+\right)$$

where $[\cdot]_+$ is the ReLU activation function, the teacher has at most $d_1^\star < d_1$ non-zero neurons (i.e., the other $d_1 - d_1^\star$ hidden neurons in the teacher to have all the incoming and outgoing weights equal to zero). Each of the teacher's weights are also in Q.

1. Calculate the probability $P_{\mathbf{w}\sim P_W}(\mathbf{w} = \mathbf{w}_\star)$.

2. Prove that

$$p_\star \triangleq P_{\mathbf{w}\sim P_W}\left(\forall \mathbf{x} : f_{\mathbf{w}}(\mathbf{x}) = f_{\mathbf{w}_\star}(\mathbf{x})\right) \geq q^{-d_0 d_1^\star - d_1}. \tag{1.1}$$

3. Show that for any constant $T > 0$, we can bound the probability of stopping time $t > T$ as

$$\lfloor T \rfloor \leq \frac{\log P(t > T)}{\log(1 - p_\star)}. \tag{1.2}$$

4. Prove the generalization bound

Theorem 1. *With probability* $(1-\eta)(1-\delta)$,

$$\epsilon < \frac{(d_0 d_1^\star + d_1)\log q + \log\frac{1}{\delta} + \log\log\frac{1}{\eta}}{N} \tag{1.3}$$

Hint: Combine the results from previous sections, using the approximations $\lfloor T \rfloor \approx T$ and $\log(1 - p_\star) \approx -p_\star$ (treat these approximations as exact), and the following basic generalization bound (which we learned in class)

Theorem 2. *For any* $f \in |\mathcal{F}|$, *with probability* $1 - \delta$,

$$\epsilon \triangleq \mathbb{P}_{\mathbf{x}}\left(f_{\mathbf{w}}(\mathbf{x}) \neq f_{\mathbf{w}_\star}(\mathbf{x})\right) < \frac{\log|\mathcal{F}| + \log\frac{1}{\delta}}{N}. \tag{1.4}$$

5. Is the bound in eq. 1.3 better than the bound in eq. 1.4 in which $\mathcal{F} = \left\{f_{\mathbf{w}} : \mathbf{w} \in Q^k\right\}$ is the student hypothesis class (in which each parameter can have one of q values)? Explain and ignore the (negligble) $\log\log\frac{1}{\eta}$ term.

2. Sampling in SGD (35 pts)

Let $\mathcal{L}$ be a β-smooth loss function which can be decomposed into N parts $\ell^{(n)}$

$$\mathcal{L}(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N} \ell^{(n)}(\mathbf{w}) \ .$$

Recall that in GD, in each step we update the parameters $\mathbf{w}$ according to the full batch gradient

$$\mathbf{g} = \frac{1}{N}\sum_{n=1}^{N} \mathbf{g}_n \ ,$$

where N is the total number of samples and $\mathbf{g}_n = \nabla \ell^{(n)}(\mathbf{w})$ is the per-sample gradient. In SGD we estimate the gradient in each step using randomly chosen $\mathbf{g}_n$ samples. Gordon, a student in the deep learning course, suggested to generalize SGD, and update the parameters in each step with a different gradient estimate

$$\Delta \mathbf{w} = \eta \hat{\mathbf{g}},$$

for some "reasonable" gradient estimate, for which $\|\mathbb{E}\hat{\boldsymbol{g}}\| = \|\boldsymbol{g}\|$, $\boldsymbol{g}^\top \mathbb{E}\hat{\boldsymbol{g}} > 0$ (i.e., the gradient estimate has an acute angle with the true gradient) and $\mathbb{E}\left[\|\hat{\boldsymbol{g}}\|^2\right] \neq 0$.

First, Gordon wants to find the optimal η in each step. To do this, he recalls from the lecture that, for β-smooth functions,

$$\mathcal{L}(\mathbf{w} - \Delta\mathbf{w}) \leq U(\eta) \triangleq \mathcal{L}(\mathbf{w}) - \eta \boldsymbol{g}^\top \hat{\boldsymbol{g}} + \frac{1}{2}\eta^2 \beta \hat{\boldsymbol{g}}^\top \hat{\boldsymbol{g}}$$

(1) What is the optimal η_* which minimizes the upper bound in expectation, i.e. $\mathbb{E}U(\eta)$? What is $\mathbb{E}U(\eta^*)$? Write the solutions as a function of $\boldsymbol{g}^\top \mathbb{E}\hat{\boldsymbol{g}}, \mathbb{E}\left[\|\hat{\boldsymbol{g}}\|^2\right]$ and β.

(2) From this result, Gordon understands that he should aim to choose a gradient estimate $\hat{\mathbf{g}}$ which unbiased, i.e. $\mathbb{E}[\hat{\mathbf{g}}] = g$ and which minimizes the variance $\mathrm{Var}(\hat{\mathbf{g}}) \triangleq \mathbb{E}\|\hat{\mathbf{g}} - \mathbb{E}\hat{\mathbf{g}}\|^2$. Explain why Gordon came to this conclusion based on section 1. Hint: Recall that the Cauchy-Shwartz inequality $\left|\mathbf{u}^\top \mathbf{v}\right| \leq \|\mathbf{u}\| \|\mathbf{v}\|$ is obtained in equality when $\mathbf{u} = \alpha \mathbf{v}$ for some constant α.

Next, Gordon chooses to use specifically

$$\hat{\boldsymbol{g}} = \frac{1}{N}\sum_{n=1}^{N} c_n \mathbf{g}_n I_n$$

where c_n are some constants and I_n is a set of 0/1 indicator functions which are sampled according to some distribution, such that $\sum_{n=1}^{N} I_n = 1$ and $p_n = \mathbb{E}I_n$.

(3) From the analysis above, Gordon wants to make sure $\hat{\mathbf{g}}$ is an unbiased estimate of the true gradient $\mathbf{g}$, i.e. $\mathbb{E}[\hat{\mathbf{g}}] = g$, where the expectation is over $\{I_n\}_{n=1}^N$ only. How should we choose c_n for this to hold?

From now on, assume c_n is selected as above to make sure $\mathbb{E}[\hat{\mathbf{g}}] = g$.

(4) Show that in this case

$$\mathrm{Var}\left(\hat{\mathbf{g}}\right) \triangleq \mathbb{E}\left\|\hat{\mathbf{g}} - \mathbb{E}\hat{\mathbf{g}}\right\|^2 = \frac{1}{N^2}\sum_{n=1}^{N}\frac{1-p_n}{p_n}\left\|\mathbf{g}_n\right\|^2$$

(5) From the analysis above, Gordon wants to make sure this variance is minimized. How should we choose p_n to do so? Hint: Construct a Lagrangian with normalization constraint for p_n.

(6) How does this sampling method compare to SGD and GD, in terms of the computational resources?

3. Neural Architectures: Mixture of experts (35 pts)

A common operation in deep learning is a "Mixtures of Experts" (MoE). This operation receives M "experts", i.e. models $\{f_{\boldsymbol{\theta}_m}(\mathbf{x})\}_{m=1}^{M}$, each with parameter $\boldsymbol{\theta}_m$, and selects the "top K" of these models.

We first define $\sigma(x) = 1/(1+e^x)$ as the sigmoid activation (for vector input, applied element-wise), $\boldsymbol{g}(\mathbf{s}) = \mathbf{s}/\|\mathbf{s}\|_1$ as the normalization operation, and $\mathrm{TopK}(\mathbf{v})$ as operation that zeros out all components in $\mathbf{v}$ which do not belong to the top K values in $\mathbf{v}$. For example, $\mathrm{Top2}((7,3,2,1,5,6,6)) = (7,0,0,0,0,6,6)$, and $\mathrm{Top2}((7,3,2,1,5,7,6)) = (7,0,0,0,0,7,0)$).

To implement an MoE, DeepSeekV3 used the following model, for $\mathbf{x} \in \mathbb{R}^d$ and $\mathbf{W} \in \mathbb{R}^{M\times d}$:

$$\mathrm{MoE}(\mathbf{x}) = \sum_{m=1}^{M} g_m\left(\mathrm{TopK}\left(\sigma(\mathbf{W}\mathbf{x})\right)\right) f_{\boldsymbol{\theta}_m}(\mathbf{x})$$

(a) Qualitatively, why and when can this be useful for improving computational efficiency during inference vs. just using M experts $\sum_{m=1}^{M} f_{\boldsymbol{\theta}_m}(\mathbf{x})$? Why is this $\mathrm{MoE}(\mathbf{x})$ better then just using K experts (i.e., $\sum_{k=1}^{K} f_{\boldsymbol{\theta}_k}(\mathbf{x})$)?

(b) $\mathrm{TopK}(\mathbf{v})$ is a discontinuous function. Where are the discontinuities?

(c) Why is the case that $K = 1$ especially problematic for optimization? Hint: What happens to $g_m(\mathrm{TopK}(\sigma(\mathbf{W}\mathbf{x})))$ in this case?

(d) For the case that $K > 2$, calculate the gradients $\nabla_{W_{i,j}}\mathrm{MoE}(\mathbf{x})$ and $\nabla_{\boldsymbol{\theta}_i}\mathrm{MoE}(\mathbf{x})$ ($i \in [1,M], j \in [1,d]$) through the layer (in the areas where $\mathrm{TopK}(\mathbf{v})$ is continuous) as a function of $\mathbf{W}$,$\mathbf{x}$ and $f_{\boldsymbol{\theta}_m}(\mathbf{x})$.

(e) During training, does MoE also help to improve computational efficiency?

(f) Why use the sigmoid activation $\sigma(x)$ instead of the common e^x (which is typically used in softmax) in the case of large $\|\mathbf{W}\|$?